

MaREA: Multi-class Random Forest for Automotive Intrusion Detection

Danilo Caivano¹, Christian Catalano², Mirko De Vincentiis¹, Alfred Lako³,
and Alessandro Pagano¹

¹ University of Bari Aldo Moro, Italy

{danilo.caivano, mirko.devincentiis, alessandro.pagano}@uniba.it

² University of Salento, Lecce

christian.catalano@unisalento.it

³ Polytechnic University of Tirana, Albania

alfred.lako@fin.edu.al

Abstract. The technology inside modern vehicles is rapidly growing and poses newer security risks, as vehicle communication protocols are not yet fully secured and vulnerable to attacks. Consequently, the implementation of automotive cybersecurity systems has gained more attention. Controller Area Network (CAN) is one of the most studied communication protocols in the literature and lacks inherent cybersecurity measures. Several works proposed Intrusion Detection Systems (IDSs) using Machine Learning (ML) and Deep Learning (DL) algorithms to identify attacks on the CAN bus. Exploiting ML or DL techniques in a multi-class approach makes it possible to know the attack typology and to support developers' decisions to integrate concrete design methods in the software automotive development life-cycle. However, most automotive IDSs are tested on data sets that contain raw CAN messages without the possibility of decoding these messages to understand how the attack was generated. Based on these gaps, a Multi-class Random Forest for Automotive Intrusion Detection (MaREA) is presented, and a new Synthetic Automotive Hacking Dataset (SA-Hacking Dataset) is generated with a Database for CAN (DBC) file. First, the model is validated on the Car-Hacking dataset and compared with two other works in the literature that used the same classifier and dataset for the multi-class approach. Then, the Random Forest model is tested by concatenating the Survival Analysis Dataset and the SA-Hacking Dataset. The proposed approach presented better-quality results for both the Car-Hacking dataset and the aforementioned concatenated dataset.

Keywords: Automotive · Artificial Intelligence · Software Engineering · IDS.

1 Introduction

The identification of cyber attacks in the automotive industry is growing faster as modern vehicles come integrated with potentially vulnerable entry points such

as Advanced Driver Assistance Systems (ADAS), Infotainment systems, OBD-II ports, and Bluetooth. It is necessary to preserve the confidentiality, integrity and availability of these systems to ensure the safety of people [8,9,13] and this is also considering the increase in projects within smart cities involving smart mobility and autonomous driving [7]. They communicate with components of the Electronic Control Unit (ECU) using different protocols. The controller area network (CAN) is the most widely used communication protocol for transmitting information between ECUs and other buses connected through different gateways [11].

Although CAN is highly reliable and resistant to electromagnetic interference [39], it has been proven to have software vulnerabilities [26,28,21]. In particular, the protocol does not implement encryption and authentication techniques, making it vulnerable to several attacks [18]. Tencent Keen Security Research Lab showed critical vulnerabilities for Tesla, Mercedes and Lexus branded cars [1,2,4,3]. For example, in [1], they found that on the CAN bus, the head unit transmitted the Passphrase and Service Set Identifier (SSID) of Wi-Fi through the CAN bus as plain text to the T-Box component. In [3], researchers, while exploiting the ADAS system of a Tesla Model S, were able to craft an image to interfere and return improper results using an adversarial attack.

Several works in the literature exploit Intrusion Detection System-based techniques to prevent attacks on the bus. However, many of these approaches use binary classification without knowing the type of attack. Additionally, automotive classification models are fitted with data sets [39] that contain the same attacks (such as denial of service, fuzzing and spoofing). Furthermore, without the Database for CAN (DBC) file, the raw messages cannot be interpreted to understand the type of attack, except for reverse engineering techniques [27,17]. In addition, it is possible to reengineer the software ECUs in order to deploy an IDS model to identify attack CAN. For example, components embedded in NVIDIA [32] could be used as ECUs because they provide high computational resources and energy efficiency.

With these considerations, the paper presents the following contributions:

- The proposition of a multi-class approach using a Random Forest to detect attack injections on a CAN bus. The proposed model (MaREA, Multiclass Random Forest for Automotive Intrusion Detection) was tested in two steps: in the first step, comparing it with existing classifiers in the literature [22,25] using Random Forest and on the same dataset, Car Hacking Dataset, while in the second phase with the Survival Analysis Dataset and the Synthetic Automotive Dataset.
- The generation of a new synthetic automotive hacking data set with a custom DBC file that contains normal and fabrication attack messages. The fabrication attack was carried out based on the work of [19] in which an adversary forged a CAN message to override any periodic messages sent by a legitimate ECU.

The paper is organized as follows. Section 2 describes the related works; Section 3 presents the background of the CAN protocol and the Database for CAN

necessary to understand the generated datasets and the dataset’s attributes; Section 4 introduces the Car-Hacking Dataset, the Survival Analysis Dataset and how the SA-Hacking Dataset has been generated; Section 5 explains the experimental phase and the proposed approach describing the preprocessing phase and how the experiments are conducted, concluding with the evaluation of the results; finally, Section 6 presents the conclusion and future works.

2 Related Work

With the adoption of sophisticated components such as Advanced Driver Assistance Systems (ADASs) and Infotainment Systems, vehicles are becoming a target to demonstrate that these components are vulnerable. Miller and Valasek [29,30] showed that vehicles are vulnerable to cyberattacks. They could remotely control a Jeep Cherokee by exploiting vulnerabilities: getting the vehicle brake, moving the steering wheel, and turning on the radio. These works have led researchers to identify new attacks and defense techniques such as IDSs, Vehicle-SOC [12], or Quantum Artificial Intelligence to support developers’ decisions to integrate concrete design methods in the automotive development life-cycle [20,36]. In particular, researchers have developed new IDSs based on the protocols used for in-vehicle networks [39]. Security solutions for IDSs in the vehicle field are focused on analyzing the messages exchanged between the ECUs that used the CAN protocol. The IDS approaches used Deep Learning (DL) or classical Machine Learning (ML) approaches. In [22], the authors compared different classical ML algorithms RF (random forest), K-NN (k-Nearest Neighbors) and XGBoost (Extreme Gradient Boosting) using the CAN-Intrusion-Dataset (also called Car-Hacking Dataset). Experiments have shown that the RF outperforms the KNN and XGBoost with an accuracy of 97%. Yang et al. [40] conducted similar work that considered the Decision Tree (DT), RF, Extra Trees (ET) and XGBoost and then an ensemble method using these models. The models were evaluated using two datasets: Car-Hacking Dataset and CIC-IDS2017. The second data set is unrelated to the automotive field because it is used for security testing and malware prevention (Web-Attack, Botnets, and Distributed Denial of Service (DDoS)). The authors combined the model that obtained the best performance for the stacking method. RF, DT, and ET. The RF obtained the best metrics compared to DT and ET. Kalkan and Sahingoz [25] proposed an IDS using different ML algorithms (RF, Bagging Tree, Adaptive Boosting, Naive Bayes, Logistic Regression) and an Artificial Neural Network tested on the Car Hacking Dataset. The authors perform a binary and a multiclass classification. Alfardus and Rawat [6] used RF, KNN, and SVM as traditional ML algorithms and a multilayer perceptron (MLP) as a Deep Learning algorithm. Using the Car-Hacking Dataset, the authors obtained 100% precision with DoS, Gear, and RPM attacks. Instead, with the Fuzzy, the best results were obtained with RF and SVM. Furthermore, Han et al. [23] created a dataset called *Survival Analysis Dataset* that collects three CAN attacks: *Flooding*, *Fuzzy*, and *Malfuction*. The authors created an algorithm to determine whether the CAN

message is not an attack by comparing the rate of the CAN ID with the ground truth. Using the same dataset, in [24], the authors proposed a deep learning model called Long Short-Term Memory (LSTM) in binary and multiclass approach. The researchers obtained better results in the binary approach than in the multiclass. In the survey by Rajapaksha et al. [34], the authors report a list of benefits and drawbacks considering the different ML algorithms used as IDS. The RF has better learning ability for small data, is able to train quickly, and has high model explainability. On the other hand, the drawback is that it has low accuracy compared to deep learning models, but it has been confirmed in the literature that RF obtained good accuracy [6,25]. Moreover, the drawback of the DL models is that they require high data to be fitted, and the datasets in the automotive field are extremely unbalanced (see Section 4). Taking into account other state-of-the-art machine learning models, such as SVM, it has better learning with small samples but is sensitive to kernel function parameters [34]. Moreover, in [31] the authors report a comparative analysis using different traditional machine learning algorithms such as RF and SVM. The results show that the RF obtained better results than the SVM.

Therefore, considering the related work and the considerations given above, RF was used as a traditional ML algorithm. Our model, MaREA, will be validated with [22] and [25] because the authors used a random forest in a multiclass approach with a state-of-the-art dataset. The Survival Analysis Dataset was used since both [23] and [24] dealt with this dataset. The results of the proposed methodology were compared only with [24] because it uses a multiclass approach instead of [23], which only uses binary classification.

3 Background

This section will briefly introduce the Controller Area Network (CAN) protocol and syntax in the Database for CAN (DBC). CAN [14] is the most used protocol for the in-vehicle network. The CAN is a broadcast protocol that allows ECUs to communicate and exchange messages between themselves. The protocol uses a bit-wise arbitration mechanism to avoid collisions when two or more messages are sent simultaneously. Considering two nodes, when these start the transmission of a frame, the messages with the lowest priority lose the arbitration, and the message with high priority can send the message on the bus. The priority of the message is determined by the Arbitration Field. The CAN protocol consists of different frames (Data, Remote, Error, and Overload), the most important being the *Data Frame*. The data frame is subdivided into different fields; the *identifier (ID)*, which is used for the arbitration phase; *Data Length Code (DLC)*, which is the length of the payload sent on the bus, and the *Data* which is the payload containing information about the message. Since the CAN protocol does not implement encryption or authentication mechanisms, it is vulnerable to different attacks such as denial of service, eavesdropping, and message injection [28,15]. In this paper, three different attack scenarios have been considered because they are the most analyzed in the literature: **Denial-of-Service**, an attacker sends

high priority messages at a certain frequency rate to suspend communication between the ECUs [38]; **Fuzzy**, an attacker can craft malicious messages by injecting random ID or data into the CAN bus [38]; **Malfunction**, an attacker uses a specific ID to craft malicious messages by sending random data with a fixed value to cause a change in vehicle behavior [23].

4 Datasets

This section introduces the datasets used to evaluate the Random Forest. The Car Hacking Dataset, Survival Analysis Dataset, and the generated dataset called SA (Synthetic Automotive) Hacking Dataset are described, and the data attributes present in each are: *Timestamp*, is the recorded time; *CAN ID*, the message identifier in hexadecimal form; *DLC*, indicates the number of bytes from 0 to 8; *DATA*, the payload in hexadecimal form from DATA[0]-DATA[7]; *Flag*, **T** represents an injected message while **R** a normal message.

(i) Car-Hacking Dataset. Car hacking⁴ [35,37] was published by the Hacking and Countermeasure Research Lab (HCRL) containing real CAN messages from a Hyundai YF Sonata car captured using an OBD-II port. The authors of the data set performed three attacks: Denial of Service (DoS), Fuzzy, and Spoofing. The spoofing attack uses the same methodology as the malfunction attack (see 3). The Car Hacking Dataset is subdivided into four datasets: DoS, Fuzzy, Spoofing RPM, and Spoofing Gear, each containing normal and attack messages. Table 1 (a) shows the number of normal and attack messages for each dataset.

(ii) Survival Analysis Dataset. The Survival Analysis Dataset⁵ [23] consists of Flooding, Fuzzy, and Malfunction attacks conducted on three vehicles: HYUNDAI YF Sonata, KIA Soul, and CHEVROLET Spark. In the flooding attack, an ECU overpopulates the CAN bus sending a large number of messages with the identifier set to *000*. In the fuzzy attack, the authors randomly generated IDs ranging from *0x000* to *0x7FF* every 0.0003 seconds. In the malfunction attack, a specific identifier was chosen for each vehicle type: *0x316* for the HYUNDAI YF Sonata, *0x153* for the KIA Soul and *0x18E* for the CHEVROLET Spark. Table 1 (b) shows the number of normal and attack messages for the three datasets contained in the KIA Soul Dataset.

(iii) SA-Hacking Dataset. Cho and Shin [19] considered three attack scenarios on in-vehicle networks: Fabrication, Suspension, and Masquerade. Considering this, in this phase of the research work, only the fabrication attack is considered, where an adversary forged the identifier, DLC, and data of a specific CAN identifier message [10]. As reported by Cho and Shin, the objective of this attack is to override any periodic messages sent by a legitimate ECU. The SA-Hacking Dataset was generated on Ubuntu 20.04 with a virtual CAN interface using SocketCAN [5], a set of open source CAN drivers. The attack phase was simulated on the basis of the fabrication attack of Cho and Shin [19]. Six threads

⁴ <https://ocslab.hksecurity.net/Datasets/car-hacking-dataset>

⁵ <https://ocslab.hksecurity.net/Datasets/survival-ids>

Table 1: Overview of messages contained in the datasets.

			KIA Soul			Number of messages	%	
(a) Car-Hacking dataset.			Flooding		Normal: 148,760	81.78		
					Attack: 33,141	18.22		
			Fuzzy		Normal: 210,178	84.07		
					Attack: 39,812	15.93		
			Malfunction		Normal: 166,035	95.73		
					Attack: 7, 401	4.27		
			(b) Kia Soul dataset.					
			SA-Hacking				Number of messages	%
(a) Car-Hacking dataset.			Fabrication		Normal: 1,322,823	82.82		
					Attack: 274,400	17.18		
			(c) SA-Hacking dataset.					

generate genuine data regarding the aforementioned bullet list of messages. Instead, a thread is used to generate attack messages. The WHEEL_SPEED (vehicle speed) and the STEER (steering angle) messages are chosen to create the fabrication attack. Every 10 seconds, 100 injected messages are sent without using the sleep function. Furthermore, each of these 100 messages is generated randomly using the *random* function⁶. Table 1 (c) shows the normal and attack messages generated in about three hours. Values are saved into a Comma Separated Values (CSV) file with attributes: Timestamp, CAN DEC (the CAN ID in decimal form), CAN ID (the CAN ID in hexadecimal form), DLC, DATA, and flag. In the Generated Dataset, the value of the attribute Flag is 0 for normal messages and 1 for the fabrication attack messages.

5 Proposed approach

5.1 Experimental phase

MaREA⁷ was tested with two different approaches. The first approach consists of validating MaREA with [22] and [25] that performed a multiclass classification with the Random Forest using the Car-Hacking Dataset to prove that our system outperformed the identified works in the literature.

In the second approach, to evaluate MaREA with a new type of attack and determine whether it is better than the classifier being analyzed, the Survival Analysis Dataset and the SA Hacking Dataset were concatenated to obtain a new dataset (Table 2 (b)). The results obtained from MaREA were compared with [24], where the authors evaluated long-term memory (LSTM) for multiclass classification using the Survival Analysis Dataset.

⁶ <https://docs.python.org/3/library/os.html>

⁷ <https://github.com/SERLABUniba/MaREA>

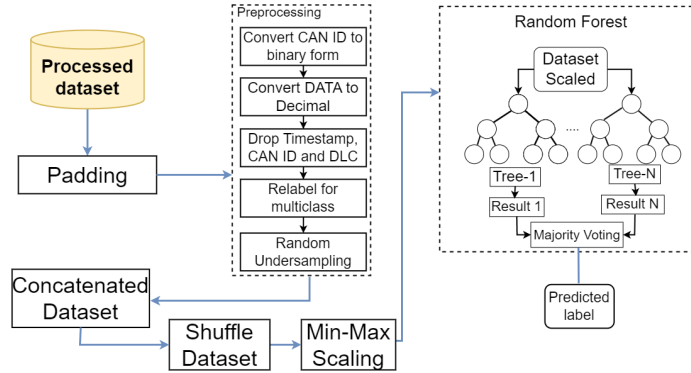


Fig. 1: An overview of the proposed methodology.

Table 2: Number of messages after the under-sampling is performed.

Label	Number messages	%	Label	Number messages	%
Normal	2,331,517	50.00	Normal	354,754	50.00
DoS	587,521	12.60	Flooding	33,141	4.67
Fuzzy	491,847	10.55	Fuzzy	39,812	5.61
Spoofing Gear	597,252	12.81	Malfunction	7,401	1.04
Spoofing RPM	654,897	14.04	Fabrication	274,400	38.67

(a) Under-sampling Car-Hacking dataset.

(b) Under-sampling Concatenated dataset.

5.2 Experimental setting

All the experiments were carried out on an Intel Core i7-11800H processor with 32 GB of RAM using Python 3 and the Scikit-learn library [33].

A Random Forest classifier [16] was used consisting of N decision trees, each of which will predict a class with majority vote. The class with the most votes becomes the model's prediction. Figure 1 shows an overview of the proposed methodology. The processed data set consists of the Car-Hacking, Survival Analysis, and SA-Hacking data set, where each of them is considered and processed individually. The padding process adds a '00' value at the end of the DATA attribute to remove any "Not a Number" (NaN) entries since all DATA attributes do not have a length of eight bytes. The padding algorithm takes the value contained in the DLC attribute and, if less than eight, adds the '00' values until it does not reach the length of eight. For example, if the DLC is four, to reach the length of eight, the algorithm will add '00' in the attribute from D4 to D7.

After padding, the preprocessing phase is performed, where the CAN IDs are transformed from hexadecimal to binary representation, obtaining eleven features. To clarify, the binary representation indicates that each CAN ID is first converted from hexadecimal to decimal form and then transformed into 0

and 1 because, for the standard identifier, it is a fixed 11-bit binary value. The DATA features were transformed from hexadecimal to decimal representation. After these phases, the following attributes were removed: *Timestamp* because MaREA does not perform a time series analysis; *CAN ID* was also removed because the model uses the binary form of the CAN ID and not the hexadecimal values; *DLC*, as the features are in a specific range from 0 to 8, it does not provide useful information for the model.

As we are conducting a multiclass, the flag data are relabeled. For the Car Hacking Dataset, the labels were transformed by mapping T with 1 for DoS, 2 for Fuzzy, 3 for Spoofing Gear, 4 for Spoofing RPM, and R with 0 for normal messages. The labels of the Survival Analysis Dataset are transformed by mapping T with 1 for Flooding, 2 for Fuzzy, 3 for Malfunction, and R with 0 for Normal messages. The label for the normal message does not change in the SA-Hacking Dataset. Instead, the attack label is transformed into 4. For each dataset, a random under-sampling was performed to resolve the class imbalance, and finally, all processed datasets were concatenated. Table 2 (a) shows the number of messages after undersampling for the Car Hacking data set, and Table 2 (b) shows the number of messages after undersampling for the concatenated Survival Analysis Dataset and the SA-Hacking Dataset. Before scaling the features, the examples are shuffled to ensure that the model does not learn any unintended pattern from the order of the data. Finally, since the features are not in a specific range, they are scaled in a range of $[0, 1]$ using the MinMaxScaler function⁸. The hyperparameters of the RF were evaluated using a grid search method by considering every combination of the values. Grid search was performed with 5-fold cross-validation using GridSearchCV⁹. Table 3 shows the values considered for the hyperparameters tuning for MaREA considering all datasets and the selected values for the Car-Hacking and Concatenated datasets.

After hyperparameters were adjusted, the concatenated data set was split into 70% for the train and 30% for the test.

Table 3: Hyperparameters considered and the values selected by the grid search for MaREA considering the Car-Hacking and Concatenated dataset.

Hyperparameters	Considered	Car-Hacking dataset	Concatenated dataset
n_estimators	9, 20, 50, 100	9	100
max_features	sqrt, log2, None	sqrt	sqrt
max_depth	10, 15, 20, 25, 30, 40, None	20	40
min_samples_split	2, 5, 10	2	5
min_samples_leaf	1, 2, 4	1	1

⁸ <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>

⁹ https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

Table 4: Comparison with other approaches that use Random Forest and the Car-Hacking Dataset as a testbed.

Reference	Accuracy	Precision	Recall	F1-Score
[22]	0.94	0.93	0.93	0.93
[25]	0.97	0.90	0.99	0.94
MaREA	0.99	0.99	0.99	0.99

Table 5: Comparison with a multi-class approach using the Survival Analysis Dataset as a testbed. The fabrication attack is reported only for MaREA. The precision score is not included in [24].

Reference	Model	Label	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
[24]	LSTM	Normal	0.9971	N/A	1.0	0.9994
		Flooding		N/A	1.0	1.0
		Fuzzy		N/A	0.9469	0.9701
		Malfunction		N/A	0.9697	0.9087
MaREA	Random Forest	Normal	0.9997	0.9999	1.0	0.9999
		Flooding		1.0	1.0	1.0
		Fuzzy		1.0	0.9956	0.9978
		Malfunction		0.9959	1.0	0.9979
		Fabrication		0.9994	1.0	0.9997

5.3 Evaluation Results

Precision, recall, F1 score, and accuracy metrics were used to compare the results. Table 4 shows the results using the Car-Hacking dataset. We can see that MaREA outperforms [22] and [25] in terms of the aforementioned metrics using a different pre-processing phase. With [25], MaREA obtains similar recall values.

Table 5 shows the results comparing MaREA with [24] using the Survival Analysis Dataset. Our model outperformed the LSTM model proposed by [24] for all types; normal, flood, fuzzy, and malfunction messages. However, both models obtained the same results for the flooding attacks, probably because the attack pattern for the flooding is easy to identify. MaREA obtained the best results for the Fuzzy and Malfunction attacks for the Recall and F1-Score metrics. In particular, the fuzzy attack is the most challenging, as the CAN IDs are randomly generated. Moreover, since MaREA is also trained with the combination of the Survival Analysis Dataset and the Synthetic Automotive Dataset, we can argue that the model generalizes properly by obtaining a low misclassification. The proposed preprocessing phase was proved to be robust in identifying CAN attacks.

6 Conclusion

This paper proposed a new synthetic automotive data set using a DBC file that can be used to generate new types of attack and an IDS called MaREA (Multi-Class Random Forest for Automotive Intrusion Detection). In the first experimental approach, MaREA was validated with two studies that used a random forest in a multiclass approach with the Car-Hacking Dataset as a testbed. Our approach achieved better results than the two random forest models. So, Random Forest was chosen as it is computationally less expansive than other Deep Learning algorithms. In the second experiment, MaREA was trained on another data set created by concatenating the Survival Analysis Dataset and the Synthetic Automotive Dataset. In this phase, the results were compared with an LSTM, where MaREA again achieved better results. In future work, we plan to create new attacks using the proposed DBC file and test the Synthetic Automotive Dataset with other models like state-of-the-art machine learning and deep learning algorithms.

Acknowledgements

This study has been partially supported by the following projects: SSA (Secure Safe Apulia, Codice Progetto 6ESURE5) and KEIRETSU (Codice Progetto V9UFIL5) funded by "Regolamento regionale della Puglia per gli aiuti in esenzione n. 17 December 2014 (BURP n. 139 suppl. del 06/10/2014) TITOLO II CAPO 1 DEL REGOLAMENTO GENERALE "Avviso per la presentazione dei progetti promossi da Grandi Imprese ai sensi dell'articolo 17 del Regolamento"; SERICS (PE00000014) under the MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU.

References

1. Experimental Security Assessment of Mercedes-Benz Cars. <https://keenlab.tencent.com/en/2021/05/12/Tencent-Security-Keen-Lab-Experimental-Security-Assessment-on-Mercedes-Benz-Cars/>, accessed: 2022-03-02
2. Experimental Security Assessment on Lexus Cars. <https://keenlab.tencent.com/en/2020/03/30/Tencent-Keen-Security-Lab-Experimental-Security-Assessment-on-Lexus-Cars/>, accessed: 2022-03-02
3. Experimental Security Research of Tesla Autopilot. <https://keenlab.tencent.com/en/2019/03/29/Tencent-Keen-Security-Lab-Experimental-Security-Research-of-Tesla-Autopilot/>, accessed: 2022-03-02
4. Exploiting Wi-Fi Stack on Tesla Model S. <https://keenlab.tencent.com/en/2020/01/02/exploiting-wifi-stack-on-tesla-model-s/>, accessed: 2022-03-02
5. Socketcan - Controller Area network. <https://docs.kernel.org/networking/can.html>, accessed: 2022-03-01
6. Alfaridus, A., Rawat, D.B.: Intrusion detection system for can bus in-vehicle network based on machine learning algorithms. In: 2021 IEEE 12th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON). pp. 0944–0949. IEEE (2021)

7. Baldassarre, M.T., Barletta, V.S., Caivano, D.: Smart program management in a smart city. In: 2018 AEIT International Annual Conference. pp. 1–6 (2018). <https://doi.org/10.23919/AEIT.2018.8577379>
8. Baldassarre, M.T., Barletta, V.S., Caivano, D., Piccinno, A.: A visual tool for supporting decision-making in privacy oriented software development. In: Proceedings of the International Conference on Advanced Visual Interfaces. AVI '20, Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3399715.3399818>
9. Baldassarre, M.T., Barletta, V.S., Caivano, D., Piccinno, A.: Integrating security and privacy in hcd-scrum. In: CHIItaly 2021: 14th Biannual Conference of the Italian SIGCHI Chapter. CHIItaly '21, Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3464385.3464746>
10. Barletta, V.S., Caivano, D., Catalano, C., De Vincentiis, M., Pal, A.: Machine learning for automotive security in technology transfer. In: Information Systems and Technologies - WorldCIST 2023 (2023)
11. Barletta, V.S., Caivano, D., Nannavecchia, A., Scalera, M.: Intrusion detection for in-vehicle communication networks: An unsupervised kohonen som approach. *Future Internet* **12**(7) (2020). <https://doi.org/10.3390/fi12070119>
12. Barletta, V.S., Caivano, D., Vincentiis, M.D., Ragone, A., Scalera, M., Martín, M.Á.S.: V-soc4as: A vehicle-soc for improving automotive security. *Algorithms* **16**(2) (2023). <https://doi.org/10.3390/a16020112>
13. Barletta, V.S., Cassano, F., Pagano, A., Piccinno, A.: New perspectives for cyber security in software development: when end-user development meets artificial intelligence. In: 2022 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT). pp. 531–534. IEEE (2022)
14. Bosch: CAN Specification Version 2.0. Robert Bosch GmbH, Postfach **50** (1991)
15. Bozdal, M., Samie, M., Aslam, S., Jennions, I.: Evaluation of can bus security challenges. *Sensors* **20**(8) (2020)
16. Breiman, L.: Random forests **45**(1), 5–32 (2001)
17. Buscemi, A., Turcanu, I., Castignani, G., Crunelle, R., Engel, T.: Canmatch: A fully automated tool for can bus reverse engineering based on frame matching. *IEEE Transactions on Vehicular Technology* **70**(12), 12358–12373 (2021)
18. Catalano, C., Afrune, P., Angelelli, M., Maglio, G., Striani, F., Tommasi, F.: Security testing reuse enhancing active cyber defence in public administration. In: ITASEC. pp. 120–132 (2021)
19. Cho, K.T., Shin, K.G.: Fingerprinting electronic control units for vehicle intrusion detection. In: USENIX Security Symposium. vol. 40, pp. 911–27 (2016)
20. De Vincentiis, M., Cassano, F., Pagano, A., Piccinno, A.: Qai4ase: Quantum artificial intelligence for automotive software engineering. In: Proceedings of the 1st International Workshop on Quantum Programming for Software Engineering. p. 19–21. QP4SE 2022, Association for Computing Machinery, New York, NY, USA
21. El-Rewini, Z., Sadatsharan, K., Selvaraj, D.F., Plathottam, S.J., Ranganathan, P.: Cybersecurity challenges in vehicular communications. *Vehicular Communications* **23**, 100214 (2020)
22. Gundu, R., Maleki, M.: Securing can bus in connected and autonomous vehicles using supervised machine learning approaches. In: 2022 IEEE International Conference on Electro Information Technology (eIT). pp. 042–046. IEEE (2022)
23. Han, M.L., Kwak, B.I., Kim, H.K.: Anomaly intrusion detection method for vehicular networks based on survival analysis. *Vehicular communications* **14**, 52–63 (2018)

24. Hossain, M.D., Inoue, H., Ochiai, H., Fall, D., Kadobayashi, Y.: Lstm-based intrusion detection system for in-vehicle can bus communications. *IEEE Access* **8**, 185489–185502 (2020)
25. Kalkan, S.C., Sahingoz, O.K.: In-vehicle intrusion detection system on controller area network with machine learning models. In: 2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT). pp. 1–6
26. Kim, K., Kim, J.S., Jeong, S., Park, J.H., Kim, H.K.: Cybersecurity for autonomous vehicles: Review of attacks and defense. *Computers & Security* **103**, 102150 (2021)
27. Marchetti, M., Stabili, D.: Read: Reverse engineering of automotive data frames. *IEEE Transactions on Information Forensics and Security* **14**(4), 1083–1097 (2019)
28. Martínez-Cruz, A., Ramírez-Gutiérrez, K.A., Feregrino-Urbe, C., Morales-Reyes, A.: Security on in-vehicle communication protocols: Issues, challenges, and future research directions. *Computer Communications* **180**, 1–20 (2021)
29. Miller, C., Valasek, C.: Adventures in automotive networks and control units. *Def Con* **21**(260-264), 15–31 (2013)
30. Miller, C., Valasek, C.: Remote exploitation of an unaltered passenger vehicle. *Black Hat USA* **2015**(S 91), 1–91 (2015)
31. Moulahi, T., Zidi, S., Alabdulatif, A., Atiquzzaman, M.: Comparative performance evaluation of intrusion detection based on machine learning in in-vehicle controller area network bus. *IEEE Access* **9**, 99595–99605 (2021)
32. NVIDIA: Jetson AGX Xavier Series. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-agx-xavier/>, Accessed: 07/25/2023
33. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* **12**, 2825–2830 (2011)
34. Rajapaksha, S., Kalutarage, H., Al-Kadri, M.O., Petrovski, A., Madzudzo, G., Cheah, M.: Ai-based intrusion detection systems for in-vehicle networks: A survey. *ACM Computing Surveys* **55**(11), 1–40 (2023)
35. Seo, E., Song, H.M., Kim, H.K.: Gids: Gan based intrusion detection system for in-vehicle network. In: 2018 16th Annual Conference on Privacy, Security and Trust (PST). pp. 1–6 (Aug 2018)
36. Serrano, M.A., Sánchez, L.E., Santos-Olmo, A., García-Rosado, D., Blanco, C., Barletta, V.S., Caivano, D., Fernández-Medina, E.: Minimizing incident response time in real-world scenarios using quantum computing. *Software Quality Journal* pp. 1–30 (2023). <https://doi.org/10.1007/s11219-023-09632-6>
37. Song, H.M., Woo, J., Kim, H.K.: In-vehicle network intrusion detection using deep convolutional neural network. *Vehicular Communications* **21**, 100198 (2020)
38. Stabili, D., Ferretti, L., Andreolini, M., Marchetti, M.: Daga: Detecting attacks to in-vehicle networks via n-gram analysis. *IEEE Transactions on Vehicular Technology* **71**(11), 11540–11554 (2022)
39. Venturi, A., Stabili, D., Pollicino, F., Bianchi, E., Marchetti, M.: Comparison of machine learning-based anomaly detectors for controller area network. In: 2022 IEEE 21st International Symposium on Network Computing and Applications (NCA). vol. 21, pp. 81–88. IEEE (2022)
40. Yang, L., Moubayed, A., Hamieh, I., Shami, A.: Tree-based intelligent intrusion detection system in internet of vehicles. In: 2019 IEEE global communications conference (GLOBECOM). pp. 1–6. IEEE (2019)